

CSCI 49900: Capstone

Group 8: Alex Haskel, James Lacoste, Kailun You

10/07/2026



Overview

- Our project is an interactive platform to demonstrate the mechanics of large language model inference optimization and to allow for the exploration of model behavior through internal interventions.
- The website will allow: Prompt → baseline + configurable experimental run → side-by-side outputs + metrics.
- The project requires writing code to optimize a model, and creating an interactive platform in which to test model behavior and performance.
- Alex: Backend + interventions, James: Backend + inference optimization, Kailun: Frontend.



Backend Development (Alex)



Defined initial schemas:

- **Generation Request**: format for requests coming into the API backend.
Includes nested schemas for:
 - **Generation settings** (decoding parameters)
 - **Intervention settings** (of different kinds, more nested stuff, explained later)
- **Generation Response**: format API responses will follow.
- Used **Pydantic** for schema validation and construction.

- Example:

```
class GenerationSettings(StrictModel):  
    max_new_tokens: int = Field(default=50, ge=1, le=100)  
    do_sample: bool = False  
    temperature: float = Field(default=0.8, gt=0.0, le=10)  
    top_k: int = Field(default=50, ge=0, le=100)  
    top_p: float = Field(default=0.95, ge=0.0, le=1)  
    repetition_penalty: float = Field(default=1.1, gt=0.0, le=50)  
    enable_thinking: bool = False
```

Backend Development (Alex)

Created tests for schemas

- Created **tests** to verify schemas enforce constraints appropriately at the backend level, and general correctness. Useful if things change as project develops.
- E.g.: empty prompt returns an error, nested schemas work properly, optional and required parameters are enforced properly, etc.
- Used **PyTest** as a testing framework

```
(backend) volador@DESKTOP-14I866V:~/projects/Capstone2026$ uv run pytest -v
backend/test_schemas.py::test_attention_intervention_settings_extra_fields PASSED
backend/test_schemas.py::test_request_extra_fields PASSED
backend/test_schemas.py::test_response_extra_fields PASSED
backend/test_schemas.py::test_MLP_intervention_discriminator PASSED
backend/test_schemas.py::test_attention_intervention_discriminator PASSED
===== 20 passed in 0.53s =====
```

Example:

```
def test_response_extra_fields():
    with pytest.raises(ValidationError):
        GenerateResponse(
            baseline_text= "paodpasodsd",
            experimental_text= "asdkasod2ed",
            baseline_latency_ms= 2.32,
            experimental_latency_ms= 3.32,
            extra_field=23,
        )
```



Defining Main Backend Functions (Alex)

- Defined basic **backend functions** in **Python** to:
 - **Load** model
 - **Call** model with user's parameters and prompt → **return** the model's **response**
 - **Intervene** model using **hooks**.
 - Hooks are functions allowing for the modification of intermediate activations, inputs, and weights w/o changing the model's architecture.

So far:

- Modify **attention heads**
- Modify **MLP units**
- Maybe more in the future (have some ideas)



Creating & Testing Backend API (Alex)

- Used **FastAPI** for API definition, **Uvicorn** as web server

```
app = FastAPI()

model, tokenizer = load_model("Qwen/Qwen3-0.6B")

@app.post("/generate", response_model=GenerateResponse)
def generate(request:GenerateRequest):

    prompt = request.prompt
    settings_a = request.configuration_a.generation
    settings_b = request.configuration_b.generation
    generated_text_a = generate_text(model, tokenizer, prompt, settings_a)
    generated_text_b = generate_text(model, tokenizer, prompt, settings_b)

    return GenerateResponse(
        condition_a_text= generated_text_a,
        condition_b_text= generated_text_b,
        condition_a_latency_ms=3.33,
        condition_b_latency_ms=123.123,
    )
```



Next Steps (Alex)

- Hooks work, but need to further test and develop backend model intervention logic; polish intervention generation wrappers
- Finish and test minimal version of API to incorporate the above
- Put together a first minimal demo by integrating to front end

Kailun You

Responsibility: Frontend

Currently...

01

Wireframe

Website layout design and planning.



02

Coding Language

Improvements in d3.js . Currently learning more applications of [d3.js](#) to eventually create and display certain functions.



Kailun You

Responsibility: Frontend

Planning...

01

Using Figma to create visual website and functions

02

Improving in D3.JS

03

Group meetings to change and fully define website visual and functions.

Kailun You

Responsibility: Frontend

Problem...

01 REACT conflicts with D3.JS

Both REACT and D3.JS can overwrite each other's changes and cause conflict.

Solution...

The easiest way to fix this problem is to use HTML and Javascript instead of REACT.

Quantization

- This week I focused on implementing the quantization and storing the artifact
- One line summary: Our INT8 version of Qwen3-0.6B now runs end to end in PyTorch
- I recorded all important findings, just in case we need to analyze them later on

STATUS UPDATE

The Phase 1 K tasks (everything before CUDA) are done

Task	What it produced	State
Model inventory	Config values, all 196 linear shapes, shared embedding confirmed	Done
Baseline dtype	FP16 confirmed and recorded	Done
Quantization library	Per channel INT8 quantize and dequantize, weight error report	Done
QuantizedLinear	Reference backend and model surgery, and lm_head left untouched	Done
Fidelity rungs 2-4	Per layer error, WikiText-2 perplexity, fixed prompt generations tested	Done
Tests	38 fast CPU tests and 3 real model GPU tests	All pass
Remaining	CUDA toolkit and profilers, container, benchmark harness, 16-bit baselines	Next

Each weight row becomes small integer plus one scale

1. Measure the row

$$s = \max |w| \div 127$$

The largest weight in the row sets the step size

2. Round to steps

$$q = \text{round}(w \div s)$$

Each weight becomes an integer from -127 to 127, stored in one byte

3. Scale once at the end

$$y = s \cdot \sum q \cdot x$$

Therefore we have one multiply per output, not one per weight

Linear layer bytes read per token

880.8 MB $\rightarrow$ 440.4 MB

Decode speedup ceiling, weights only

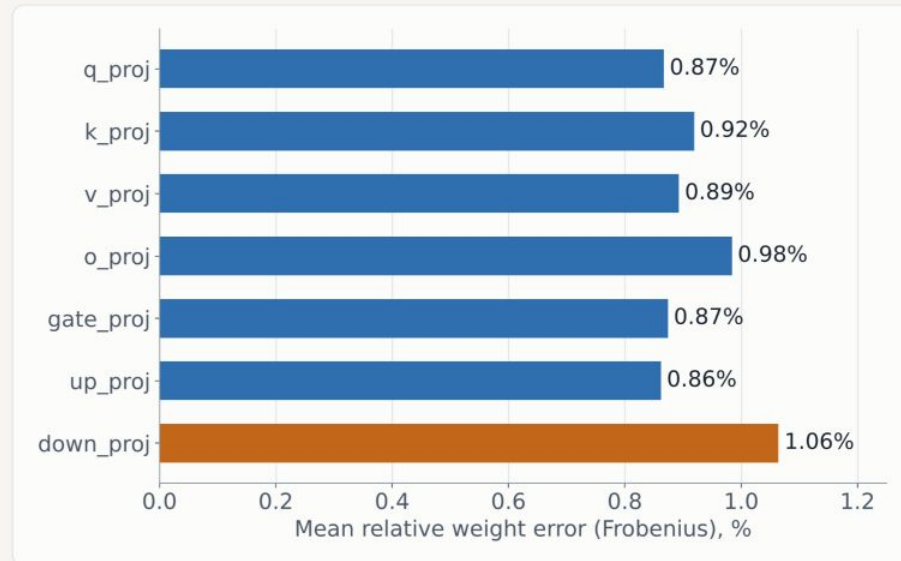
1.59×

Worst rounding error

Half a step

WEIGHT ERROR

Rounding changes the weights by about 1%



Bounded as designed

The worst error is 0.5000018 of a step; in theory would be 0.5. The extra is FP32 rounding. No weight was clipped.

down_proj is worst

9 of the 10 highest-error matrices are down_proj. SQNR ranges from 38.4 to 41.8 dB.

Scales stay FP32

Three k_proj scales fall below the smallest normal FP16 value (6.1e-5), so FP16 would lose precision there.

Some findings

- The speedup will come from memory traffic
- During “decoding” transformer models take each weight and read once per token, so time tracks # of bytes read
- The linear layers drop from 880.8 MB to 440.4 MB per token
- We still pass all the tests I ran, so the reasoning ability of the model did not appear to degrade

A few quick evaluations

```
[  
  {"id": "capital", "text": "What is the capital of France?"},  
  {"id": "arithmetic", "text": "What is 17 multiplied by 23? Show your steps."},  
  {"id": "code", "text": "Write a Python function that reverses a singly linked list."},  
  {"id": "instruction", "text": "List three fruits that are red, one per line, with no other text."},  
  {"id": "explanation", "text": "Explain how a refrigerator keeps food cold."},  
  {"id": "summary", "text": "Summarize the plot of Romeo and Juliet in two sentences."},  
  {"id": "translation", "text": "Translate 'The weather is nice today' into Spanish and French."},  
  {"id": "reasoning", "text": "A train leaves at 3:15 pm and the trip takes 2 hours 50 minutes. When does it arrive?"}  
]
```

No prompt goes from right to wrong

Prompt	INT8 differs at	BF16 differs at	Note
capital	identical	identical	Paris in all three
arithmetic	token 3	identical	Both get 391; INT8 orders the steps differently
code	identical	identical	Correct linked-list reversal
instruction	identical	identical	Baseline already lists a "red banana"
explanation	token 17	token 17	Both contain some slight factual errors
summary	token 27	identical	Both correct; INT8 ends the sentence early
translation	identical	identical	Baseline gives Spanish only
reasoning	token 101	identical	All say 5:65 pm; the baseline is wrong

NOTES: Non thinking mode was used, seeded sampling (temperature 0.7, top-p 0.8, top-k 20), 128 new tokens

Next Steps

- I need to build a more comprehensive benchmark harness for evaluations
- Research to get ready for kernel development, read through the CUDA documentation
- Work as a team on API contract and frontend